

Relazione Progetto MapReduce - libmr

Framework per l'analisi di file di testo
Laboratorio II - Università di Pisa
Luglio 2026

1 Introduzione

Il progetto consiste nella realizzazione di una libreria C denominata **libmr**, che implementa il paradigma **MapReduce** per l'analisi di file di testo su un singolo computer — a differenza delle implementazioni distribuite (come Hadoop) che operano su cluster di macchine. Il framework gestisce autonomamente la creazione dei processi, la comunicazione tramite pipe POSIX e l'orchestrazione dei thread C11, permettendo all'utente di concentrarsi esclusivamente sulla logica di *mapping* e *reducing*.

La libreria è completamente operativa e implementa tutte le funzionalità richieste dalla specifica, comprese le estensioni dell'addendum: scansione ricorsiva delle directory, chiamate multiple a `mr_start()`, file delle statistiche e funzione di hashing personalizzabile.

2 Interfaccia Pubblica

L'interfaccia pubblica è definita in `include/mr.h` e si basa su un handle opaco `mr_t`. Le funzioni principali sono:

- `mr_attr_init() / mr_attr_destroy()`: inizializzano e rilasciano la struttura di configurazione `mr_attr_t`.
- `mr_attr_set_*`(): setter per configurare numero di thread worker, dimensione delle code, file di log e funzione di hashing.
- `mr_create()`: alloca sull'heap una nuova istanza `struct mr`, copia tutti i campi da `attr` e salva le callback utente.
- `mr_start()`: avvia l'elaborazione MapReduce in modo **bloccante**, ovvero non ritorna al chiamante finché tutta l'elaborazione non è completata (mapper, reducer e scrittura dell'output). Accetta un file singolo o una directory come input.
- `mr_destroy()`: rilascia la memoria allocata da `mr_create()`.

Valori di default in `src/mr.c`: `MR_DEFAULT_MAPPER_THREADS = 1`,
`MR_DEFAULT_REDUCER_THREADS = 1`, `MR_DEFAULT_QUEUE_SIZE = 100`,
`MR_DEFAULT_LOG_FILE = "mr.log"`.

3 Architettura e Gestione dei Processi

Il framework si basa su una pipeline di tre processi distinti che comunicano tramite pipe POSIX anonime. La creazione avviene tramite `fork()` all'interno di `mr_start()`.

3.1 Creazione della pipeline

- **Setup delle pipe:** tre pipe anonime create con `pipe()`: pipe A (Main->Mapper), pipe B (Mapper->Reducer), pipe C (Reducer->Main).
- **Fork del Mapper:** il figlio duplica con `dup2(fd_A[0], STDIN_FILENO)` e `dup2(fd_B[1], STDOUT_FILENO)`, chiude i fd superflui ed esegue `mapper_run()`.
- **Fork del Reducer:** collega `fd_B[0]` a `stdin` e `fd_C[1]` a `stdout`, chiude i fd superflui ed esegue `reducer_run()`.

3.2 Gestione dei file descriptor

In ciascun processo vengono chiuse sistematicamente tutte le estremità non utilizzate. Questo è essenziale per:

- **Propagazione dell'EOF:** dopo una `fork()`, sia il padre che il figlio ereditano una copia di ogni file descriptor. Una pipe riceve EOF solo quando *tutti* i descrittori del lato di scrittura sono stati chiusi — in tutti i processi che ne hanno una copia. Se anche un solo processo dimenticasse di chiudere un fd di scrittura inutilizzato, il destinatario rimarrebbe bloccato in una `read()` infinita.
- **Prevenzione di deadlock:** senza la chiusura tempestiva, i processi attenderebbero segnali di terminazione che non verranno mai generati.

3.3 Coordinamento e terminazione

Dopo aver inviato tutti i dati su pipe A, il padre chiude `fd_A[1]`. Questo innesca una reazione a catena: il Mapper riceve EOF, chiude `stdout` (pipe B); il Reducer riceve EOF, scrive i risultati su pipe C e chiude `stdout`; il padre legge i risultati e invoca `waitpid()` su entrambi i figli.

4 Organizzazione dei Thread

Sia il Mapper che il Reducer sono multithreaded e utilizzano lo standard **C11** (`threads.h`) con `mtx_t` e `cnd_t`.

4.1 Processo Mapper

- **Thread lettore:** legge le righe serializzate da `stdin` con `readn()` e le inserisce nella coda interna.
- **Thread worker** (N istanze): estraggono le righe dalla coda ed invocano la callback `mapper` in parallelo al thread lettore, che continua a leggere da `stdin` nel frattempo. L'accesso alla coda è sincronizzato tramite `mtx_t` e `cnd_t` (implementati in `src/coda.c`): i worker si bloccano sulla condition variable quando la coda è vuota, e

riprendono appena il lettore inserisce nuove righe. Le coppie (token, valore) prodotte sono scritte su stdout protette da mutex.

- A EOF, il thread lettore chiama `queue_close()`; quando tutti i worker terminano, il processo chiude stdout per segnalare EOF al Reducer.

4.2 Processo Reducer

- **Thread lettore:** legge le coppie da stdin e le raggruppa nella tabella hash interna.
- **Raggruppamento:** a differenza del Mapper — dove i worker lavorano in parallelo al lettore — nel Reducer i worker iniziano solo dopo che il thread lettore ha ricevuto EOF e completato il raggruppamento. Questo e' necessario perche' i valori di uno stesso token possono arrivare in qualsiasi ordine: solo a input terminato si ha la certezza di aver raccolto tutti i valori associati a ciascun token. I gruppi completi vengono poi inseriti in una coda per i thread worker.
- **Thread worker** (N istanze): invocano la callback reducer e scrivono i risultati su stdout, protetti da mutex.

4.3 Coda interna

La coda thread-safe e' implementata in `src/coda.c` come buffer circolare di capacita' configurabile (`queue_size`), sincronizzato con `mtx_t` e `cnd_t` C11 secondo il modello produttore-consumatore bloccante: i consumatori attendono sulla condition variable quando la coda e' vuota; il produttore attende quando e' piena.

5 Formato dei Messaggi sulle Pipe

Protocollo binario con lunghezze esplicite, gestito da `writen()` e `readn()`.

5.1 Main -> Mapper (pipe A)

```
file_name_len (size_t)
line_number   (unsigned long)
line_len      (size_t)
file_name     (file_name_len byte, senza terminatore nullo)
line          (line_len byte)
```

5.2 Mapper -> Reducer (pipe B)

Per ogni coppia emessa dalla callback mapper, il messaggio contiene:

```
token_len      (size_t)    -- lunghezza del token
value_len      (size_t)    -- lunghezza del payload del valore
token          (token_len byte)
payload        (value_len byte, dati opachi emessi dal mapper)
```

5.3 Reducer -> Main (pipe C)

```
token_len      (size_t)
result_len     (size_t)
token          (token_len byte)
result         (result_len byte, dati opachi)
```

6 Strutture Dati

6.1 Tabella hash nel Reducer

Il raggruppamento per token avviene tramite una tabella hash con gestione delle collisioni per **chaining** (inserimento in testa a una lista concatenata). La dimensione è configurata tramite `hashtable_init()`. Ogni nodo mantiene il token e un array dinamico di strutture `mr_value_t`.

6.2 Array dei risultati nel Main

Implementato in `src/mr.c` all'interno di `mr_start()`: dopo aver ricevuto EOF da pipe C, il padre ha accumulato i risultati in un array dinamico di strutture `risultato_t` (`token_len`, `token`, `result_len`, `result`) tramite `realloc()`. L'array viene poi ordinato lessicograficamente con `qsort()` e scritto nel file di output.

7 File di Output e Sistema di Logging

7.1 Formato del file di output

File binario a record, ordinato lessicograficamente per token. Ogni record:

```
token_len  (size_t)
token      (token_len byte)
result_len (size_t)
result     (result_len byte, dati opachi)
```

7.2 Sistema di logging

Log scritto su file di testo (default: `mr.log`). Formato:

```
[timestamp] [PID] [TID] [evento] messaggio
```

La sincronizzazione tra processi avviene tramite semaforo POSIX named (`sem_open`, `sem_wait`, `sem_post`, `sem_unlink`), garantendo scritture atomiche da parte di Main, Mapper e Reducer sullo stesso file. Vengono registrati: creazione delle pipe, fork, avvio/fine dei thread, righe inviate, risultati prodotti.

8 Addendum

8.1 Scansione ricorsiva delle directory

La funzione statica `collect_files()` esplora ricorsivamente una directory con `opendir()`, `readdir()` e `stat()`. Appena viene incontrata una sottodirectory, la funzione chiama se stessa su quel percorso, entrando dentro e completando l'esplorazione prima di proseguire con le altre entry della directory corrente. I file regolari vengono aggiunti a un array dinamico. Al termine, l'array viene ordinato con `qsort()` per garantire un ordine deterministico.

8.2 Chiamate multiple a `mr_start()`

Tutte le risorse allocate da `mr_start()` — pipe, processi, semaforo, array di file, array dei risultati — sono create e distrutte all'interno della singola chiamata. La struttura `struct mr` non viene modificata tra una chiamata e l'altra: non vi rimane alcun file descriptor aperto, puntatore pendente o stato residuo che potrebbe causare comportamenti imprevedibili (interferenze) nella chiamata successiva, né memoria allocata e mai liberata (memory leak). Una sola chiamata a `mr_destroy()` è quindi sufficiente.

8.3 File delle statistiche

Al termine di ogni `mr_start()`, le statistiche vengono aggiunte in append a `statistics.txt`: percorso di input e output, righe lette, token distinti, risultati emessi e tempo totale di esecuzione in secondi misurato con `clock_gettime(CLOCK_MONOTONIC, ...)`.

8.4 Funzione di hashing personalizzabile

`mr_attr_t` espone i campi `hash` (tipo `mr_hash_t`) e `hash_arg`. Tramite `mr_attr_set_hash_function()` l'utente fornisce una funzione personalizzata. Se il campo è `NULL`, il Reducer usa la funzione di hashing di default per distribuire i token tra i thread worker.

9 Test e Validazione

Suite di test automatizzati in `tests/test_ping.c`, compilata con `make test`. Ogni test crea i file di input necessari, invoca `mr_start()`, legge l'output binario tramite la funzione `helper leggi_conteggio()` e rimuove i file temporanei.

I cinque test coprono:

- **test_file_vuoto**: file di input vuoto; verifica che l'output sia vuoto.
- **test_riga_singola**: file con una sola riga; ogni token deve avere conteggio 1.
- **test_senza_newline**: file la cui ultima riga non termina con `'\n'`. Poiché `fgets()` legge fino a EOF anche senza newline, e il mapper usa `'\n'` come delimitatore opzionale, il token finale deve risultare nel file di output con conteggio 1 — esattamente come se la riga terminasse normalmente.
- **test_chiavi_duplicate**: directory con due file contenenti la parola 'cane'; verifica che il reducer sommi i conteggi (atteso: 2).
- **test_ordine_lessicografico**: directory con tre file (`a.txt`, `b.txt`, `c.txt`) ciascuno con una parola univoca; verifica che tutti e tre i token siano presenti nell'output.

Valgrind (`--leak-check=full --track-fds=yes`) su tutti i processi ha prodotto **0 errori e 0 memory leak**. Compilazione con `-Wall -Wextra -std=c11`: 0 warning.

10 Conclusioni

libmr implementa con successo il modello MapReduce sfruttando le **primitive POSIX** — ovvero le funzioni di sistema garantite dallo standard POSIX su tutti i sistemi Unix-like: `fork()`, `pipe()`, `dup2()`, `waitpid()`, `sem_open()` e simili — e lo standard C11.

L'architettura a pipeline garantisce l'**isolamento tra le fasi di elaborazione**: ogni fase (invio dati, mapping, reducing, scrittura output) avviene in un processo separato con la propria memoria privata. I processi comunicano esclusivamente attraverso le pipe, quindi un errore in un processo non può corrompere i dati degli altri.

Il **multithreading con thread C11** permette di sfruttare più core della CPU contemporaneamente: con N thread worker su una macchina a N core, più righe (nel Mapper) o più token (nel Reducer) vengono elaborati in parallelo, riducendo il tempo totale di esecuzione rispetto a un approccio single-thread.

Il framework rispetta i requisiti di robustezza (Valgrind clean, 0 warning), correttezza (5/5 test automatizzati) e determinismo (ordinamento lessicografico garantito).
